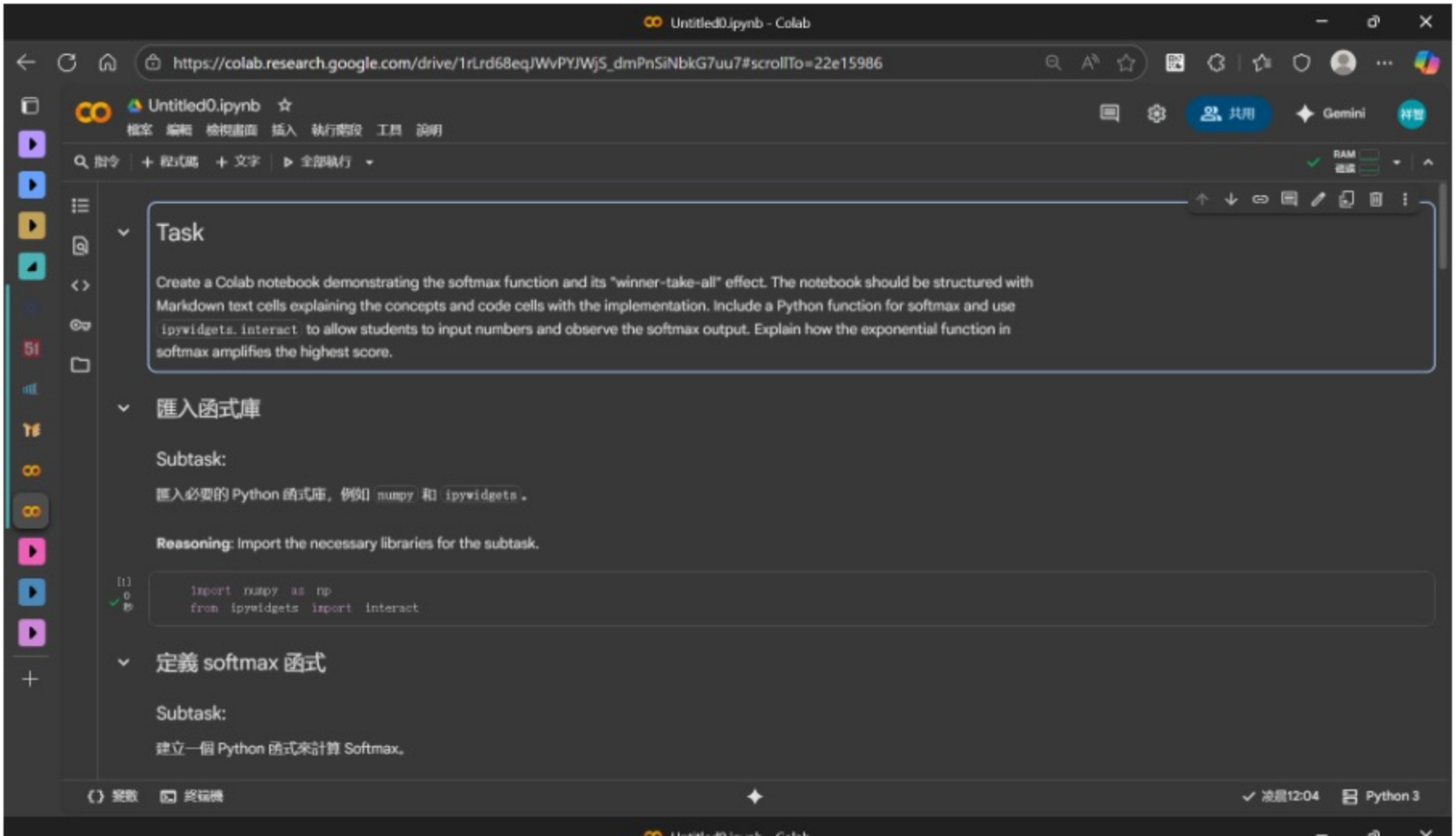
[政大109205057財政四徐祥智_Gemini]

[政大109205057財政四徐祥智_Cursor]

流程說明:

首先以本次作業提供的prompt直接在colab內建的Gemini中生成softmax模型。



Untitled0.ipynb - Colab

https://colab.research.google.com/drive/1rLrd68eqJWvPYJWjS_dmPnSiNbkG7uu7#scrollTo=22e15986

Untitled0.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

Reasoning: Define the softmax function as described in the instructions, including converting the input to a NumPy array, computing exponentials, summing them, and then normalizing.

```
def softmax(x):
    """Computes the softmax function for a given input array or list.

    Args:
        x: A NumPy array or list of numbers.

    Returns:
        A NumPy array representing the softmax probabilities.
    """
    x = np.array(x)  # Convert input to NumPy array
    e_x = np.exp(x)  # Compute exponential of each element
    return e_x / e_x.sum() # Normalize by dividing by the sum of exponentials
```

建立互動介面

Subtask:

使用 `ipywidgets.interact` 建立一個互動介面，讓學生輸入數字。

Reasoning: Define the `softmax_interactive` function and use `ipywidgets.interact` to create an interactive interface based on the instructions.

```
def softmax_interactive(input1, input2, input3):
    """Takes three numerical inputs, computes their softmax, and prints the result."""
    input_array = np.array([input1, input2, input3])
    output_softmax = softmax(input_array)
```

變數 終端機 凌晨12:04 Python 3

Untitled0.ipynb - Colab

https://colab.research.google.com/drive/1rLrd68eqJWvPYJWjS_dmPnSiNbkG7uu7#scrollTo=22e15986

Untitled0.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

```
def softmax_interactive(input1, input2, input3):
    """Takes three numerical inputs, computes their softmax, and prints the result."""
    input_array = np.array([input1, input2, input3])
    output_softmax = softmax(input_array)
    print(f"Input values: {input_array}")
    print(f"Softmax output: {output_softmax}")

interact(softmax_interactive, input1=(-10.0, 10.0, 0.1), input2=(-10.0, 10.0, 0.1), input3=(-10.0, 10.0, 0.1));
```

```
input1 0.00
input2 0.00
input3 0.00
Input values: [0. 0. 0.]
Softmax output: [0.33333333 0.33333333 0.33333333]
```

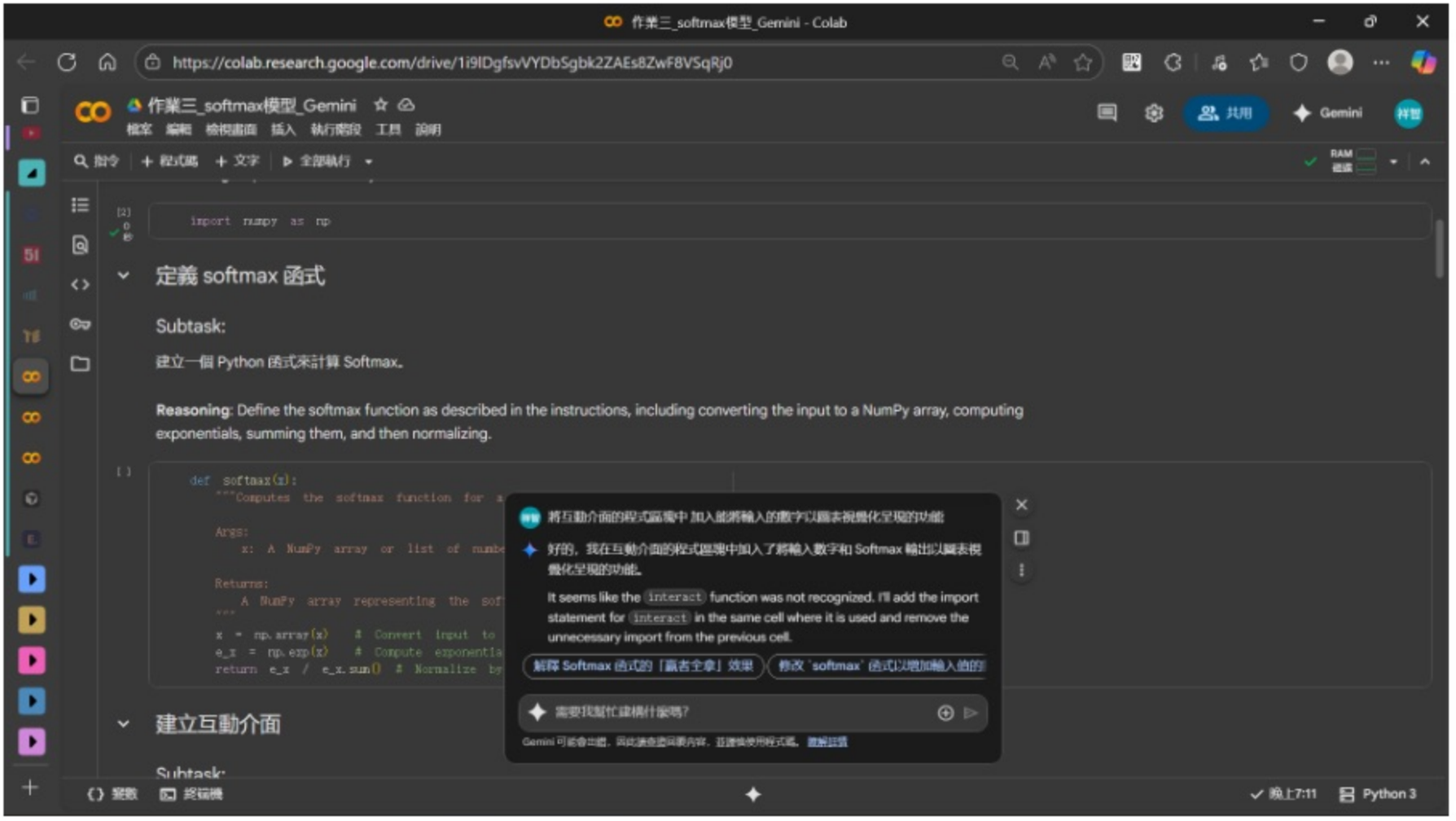
顯示結果

Subtask:

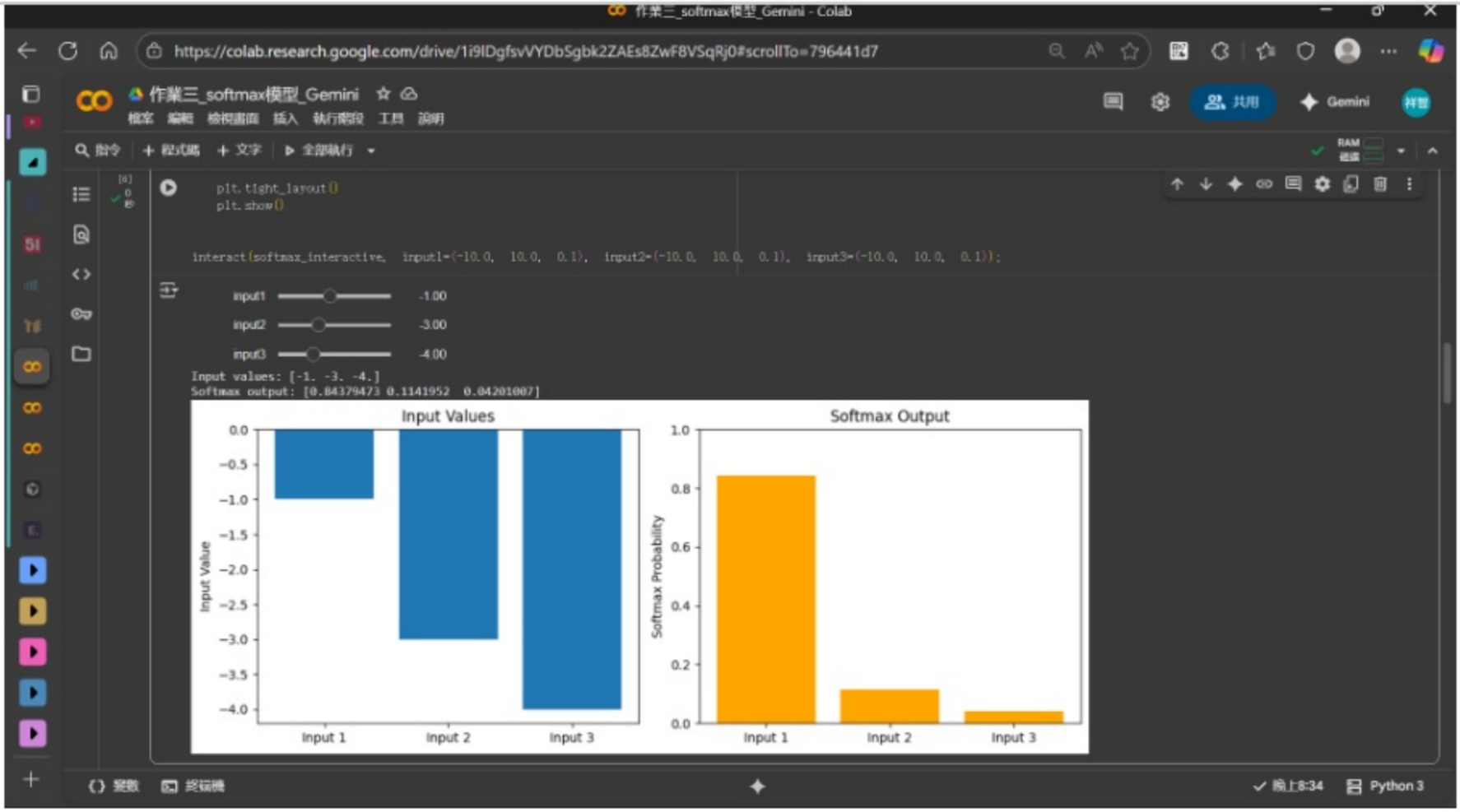
在互動介面中顯示輸入數字經過 Softmax 處理後的結果。

說明 softmax 效應

後覺得純粹只有數字不夠直觀，於是再下prompt生成互動式視覺化圖表。







因上課時老師已詳細說明過softmax的函數概念，作業給的提示範例也蠻完整的，我認為只在一個AI模型中生成函數有點單調，於是又用一樣的提示詞嘗試了Cursor中以GPT-5為主模型生成的softmax函數。



作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=h6IRTN0nTzIz

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

1) 標題與導覽

Softmax「贏者通吃」(Winner-Take-All) 直觀示範

本筆記本帶你理解為什麼 softmax 會有「贏者通吃」的效果：由於使用了指數函式，較大的分數會被放大得更厲害，導致機率高度集中在最大者。

你可以：

- 輸入三個分數，看看經過 softmax 後的機率分佈
- 調整放大量級（scale；也可理解為溫度 temperature 的倒數），觀察機率如何變尖或變平

請依序執行每個儲存格，並依照說明進行互動與練習。

2) Softmax 概念

Softmax 是什麼?

給定分數向量 x = [x1, x2, x3]，softmax 會把它轉成機率： softmax(x)_i = exp(x_i) / Σ_j exp(x_j)

- 「贏者通吃」：若某個分數最大，exp 會把它放大更多，機率就更集中在它身上。
- 數值穩定：直接算 exp 可能溢位，常見做法是先將每個 x_i 減掉最大值 max(x)，效果等同但更穩定。
- 溫度/scale：我們也可以用 scale（相當於 1/temperature）來控制尖銳程度： softmax(scale * x)。scale 越大，分佈越尖；scale 越小，分佈越平。

```
[0]
0
    from google.colab import output
```

變數 終端機

✓ 晚上8:58 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=h6IRTN0nTzIz

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

```
[0]
✓ 0
秒
    from google.colab import output
    output.enable_custom_widget_manager()
```

Support for third party widgets will remain active for the duration of the session. To disable support:

```
[?]
✓ 0
秒
    from google.colab import output
    output.disable_custom_widget_manager()
```

3) 安裝與匯入

```
[0]
✓ 6
秒
    # 若 ipywidgets 還沒啟用，Colab 需要這行
    # 但新版 Colab 通常已內建，若遇到互動元件無法顯示再執行這行並重新啟動執行階段
    # !pip install ipywidgets==8.1.3
    !pip install ipywidgets==8.1.3
    import math
    from typing import List
    import numpy as np
    from ipywidgets import interact, FloatText, FloatSlider, HBox, VBox, Label, HTML
    import ipywidgets as widgets
    import matplotlib.pyplot as plt

    # 在 Colab 裡顯示互動式小工具
    import warnings
```

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=OVOMM8aFUVHJ

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

```
from google.colab import output
output.enable_custom_widget_manager()
```

Support for third party widgets will remain active for the duration of the session. To disable support:

```
from google.colab import output
output.disable_custom_widget_manager()
```

4) 實作 numerically stable softmax

```
def softmax(scores: List[float], scale: float = 1.0) -> np.ndarray:
    """
    Numerically stable softmax.
    - scores: 一組分數（list 或 1D array）
    - scale: 放大量級（= 1/temperature），越大越尖銳，越小越平坦。
    回傳：np.ndarray，三者機率和為 1
    """
    scores = np.asarray(scores, dtype=np.float64)
    scaled = scores * float(scale)

    # 數值穩定：先減掉最大值
    max_val = np.max(scaled)
    shifted = scaled - max_val

    exps = np.exp(shifted)
    denom = np.sum(exps)
    # 避免極端情況 denom 為 0
    if denom == 0.0 or not np.isfinite(denom):
        # 退化處理：全部給相等機率
        return np.ones_like(exps) / exps.size
```

變數 終端機

剛上8:58 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=OVOMM8aFUVHJ

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

```
def softmax(scores: List[float], scale: float = 1.0) -> np.ndarray:
    """
    Numerically stable softmax.
    - scores: 一組分數（list 或 1D array）
    - scale: 放大量級（= 1/temperature），越大越尖銳，越小越平坦。
    回傳：np.ndarray，三者機率和為 1
    """
    scores = np.asarray(scores, dtype=np.float64)
    scaled = scores * float(scale)

    # 數值穩定：先減掉最大值
    max_val = np.max(scaled)
    shifted = scaled - max_val

    exps = np.exp(shifted)
    denom = np.sum(exps)
    # 避免極端情況 denom 為 0
    if denom == 0.0 or not np.isfinite(denom):
        # 退化處理：全部給相等機率
        return np.ones_like(exps) / exps.size
    return exps / denom
```

5) 基本測試與示範

```
examples = [
```

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=OVOMM8aFUVHJ

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

指令 + 程式碼 + 文字 全部執行

共用 Gemini

5) 基本測試與示範

```
examples = [
    [0, 0, 0],
    [1, 2, 3],
    [3, 3, 3],
    [5, 0, -5],
]

for ex in examples:
    print(f"input: {ex} -> softmax: {softmax(ex)}")
```

```
input: [0, 0, 0] -> softmax: [0.33333333 0.33333333 0.33333333]
input: [1, 2, 3] -> softmax: [0.09003057 0.24472847 0.66524096]
input: [3, 3, 3] -> softmax: [0.33333333 0.33333333 0.33333333]
input: [5, 0, -5] -> softmax: [9.93262357e-01 6.69254912e-03 4.50940412e-05]
```

6) 互動式觀察：三個輸入與 scale

```
def visualize_softmax(a: float, b: float, c: float, scale: float):
    scores = [a, b, c]
    probs = softmax(scores, scale=scale)

    # 指標：熵（越小越尖銳）、最大機率與其索引
    eps = 1e-12
    entropy = float(-(probs * np.log(probs + eps)).sum())
    top_idx = int(np.argmax(probs))
    top_prob = float(probs[top_idx])

    # 顯示數值與指標
    header = HTML(value="<b>Input scores</b> → " + str(np.round(scores, 4)))
    smx = HTML(value="<b>Softmax</b> → " + str(np.round(probs, 6)) + f" (sum={probs.sum():.6f})")
    metrics = HTML(value=f"<b>Metrics</b> → entropy={entropy:.4f}, top={top_idx+1}, top_prob={top_prob:.4f}")

    # 視覺化，左邊當前 scale 的長條圖；右邊不同 scale 的機率曲線
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), dpi=120)

    # Left: bar chart at current scale
    labels = ["z1", "z2", "z3"]
    colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]
    axes[0].bar(labels, probs, color=colors)
    axes[0].set_ylim(0.0, 1.0)
    axes[0].set_title(f"Softmax at scale = {scale:.2f}")
    for i, v in enumerate(probs):
```

變數 終端機 晚上8:58 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=OVOMM8aFUVHJ

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

```
# Right: probability vs scale curve
# 使用對數空間能看出尖/平轉變，範圍可依需調整
scale_min, scale_max = 0.01, 10.0
scale_grid = np.linspace(scale_min, scale_max, 300)
prob_curves = np.array([softmax(scores, s) for s in scale_grid])  # shape: (N, 3)

axes[1].plot(scale_grid, prob_curves[:, 0], color=colors[0], label="P(x1)")
axes[1].plot(scale_grid, prob_curves[:, 1], color=colors[1], label="P(x2)")
axes[1].plot(scale_grid, prob_curves[:, 2], color=colors[2], label="P(x3)")

# 垂直線標示目前的 scale
axes[1].axvline(scale, color="#444444", linestyle="--", linewidth=1)
axes[1].set_xlim(scale_min, scale_max)
axes[1].set_ylim(0.0, 1.0)
axes[1].set_xlabel("scale (1/temperature)")
axes[1].set_ylabel("probability")
axes[1].set_title("Probability vs. scale")
axes[1].legend(loc="best", frameon=False)

plt.tight_layout()
plt.show()

display(VBox([header, smx, metrics]))


# 建立互動元件
a_box = FloatText(value=1.0, description="x1:", step=0.1)
b_box = FloatText(value=2.0, description="x2:", step=0.1)
c_box = FloatText(value=3.0, description="x3:", step=0.1)

scale_slider = FloatSlider(
    value=1.0, min=0.01, max=10.0, step=0.01,
    description="scale",
    readout_format=".2f",
    continuous_update=False,
```

變數 終端機

晚上8:58 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=OVOMM8aFUVHJ

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

```
# 建立互動元件
a_box = FloatText(value=1.0, description="x1:", step=0.1)
b_box = FloatText(value=2.0, description="x2:", step=0.1)
c_box = FloatText(value=3.0, description="x3:", step=0.1)

scale_slider = FloatSlider(
    value=1.0, min=0.01, max=10.0, step=0.01,
    description="scale",
    readout_format=".2f",
    continuous_update=False,
)

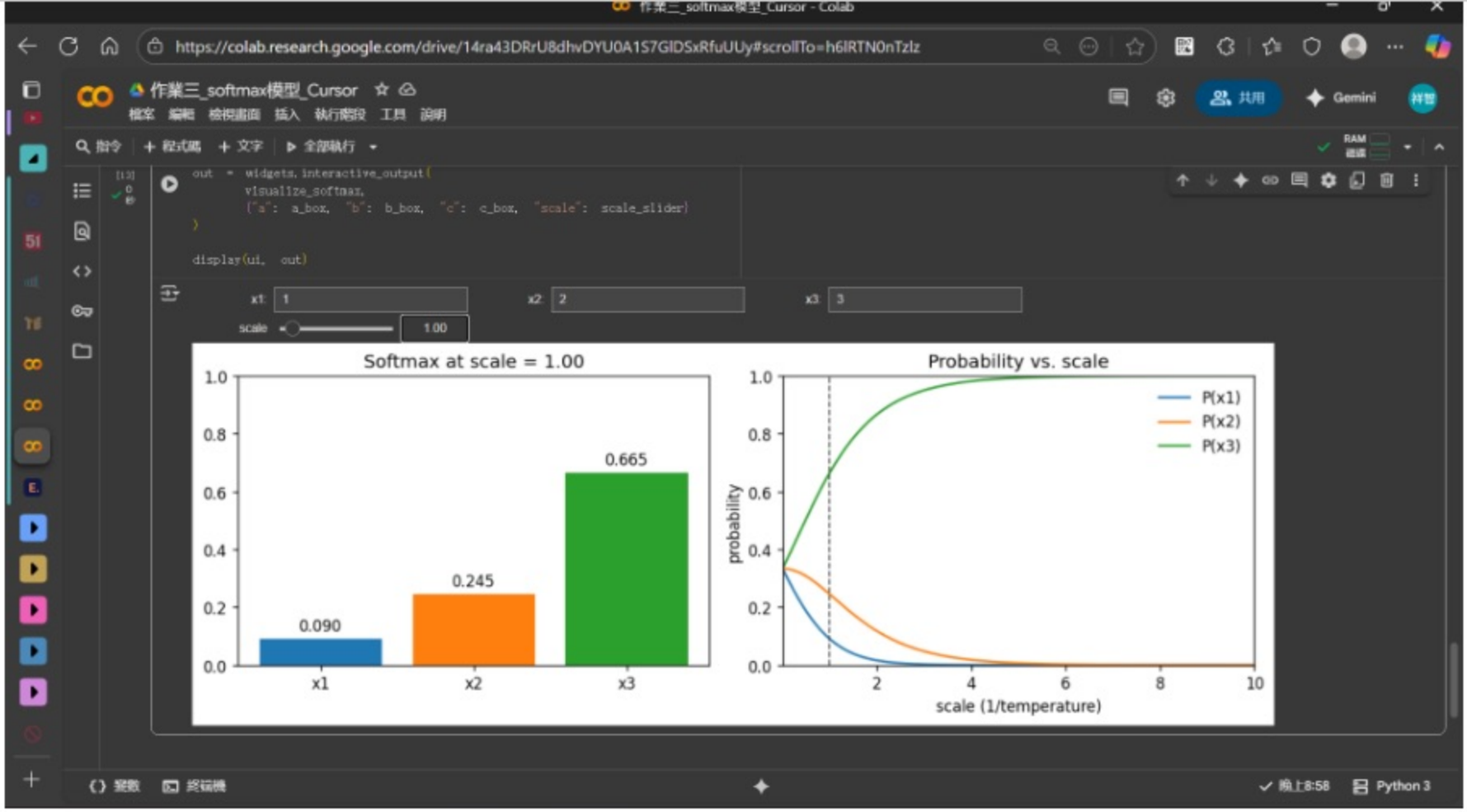
ui = VBox([
    HBox([a_box, b_box, c_box]),
    scale_slider
])

out = widgets.interactive_output(
    visualize_softmax,
    {"a": a_box, "b": b_box, "c": c_box, "scale": scale_slider}
)

display(ui, out)
```

x1: 1.1 x2: 1.2 x3: 1.3

scale 9.00



最後想到在原本的softmax函數式上做一點變化，引入scale與temperature兩個用途不相同但其實互為倒數的調整參數。



在互動式元件的程式區塊新增能產生視覺化圖表的程式功能 以方便學生檢閱分析

- 現在除了當前 `scale` 的長條圖外，右側還會顯示 `P(x1)`, `P(x2)`, `P(x3)` 隨 `scale` 的曲線與目前 `scale` 的指示線，便於比較「贏者通吃」如何隨放大量級變化。
- 也會顯示熵 `entropy`（越小表示分佈越尖）與當前最大機率的索引與數值，便於定量分析。
- 若需要把 `scale` 改成使用 temperature，將 `softmax(scores, scale=scale)` 改為 `softmax(scores, scale=1.0/max(1e-6, T))`，並把滑桿描述改為 `temperature` 即可。

在互動式元件的程式區塊新增temperature 的參數調整功能

Thought for 32s

我會在互動區塊加入「temperature」滑桿與控制模式切換（scale / temperature），兩者可二選一控制；系統會自動換算

有效 scale = 1 / temperature 並一起顯示，方便對照分析。

請將互動式觀察的「程式碼」儲存格整段以以下內容取代並執行：

```
def visualize_softmax(a: float, b: float, c: float,
    scores = [a, b, c]
```

@Add Context 8.9%

Plan, @ for context, / for commands

我們要讓學生有一個看 softmax 有那種「贏者

在互動式元件的程式區塊新增temperature 的參數調整功能

- 切換 control 為「temperature」後，系統會以 temperature 為主，並自動換算成有效 scale 顯示在圖表與指標中。
- 若想僅保留 temperature 控制，也可以刪除 `scale_slider` 與 `mode_toggle`，並在 `compute_and_visualize` 中固定以 `effective_scale = 1.0 / max(eps, temperature_value)` 計算。

可以改成能同時調整scale與temperature嗎?

Thought for 5s

好的，可以。以下是可同時調整 scale 與 temperature 的互動式程式區塊（兩者同時生效，等效 scale = scale × 1/temperature）。請用這段整體覆蓋你先前的互動區塊程式碼執行。

```
def visualize_softmax(a: float, b: float, c: float,
    scores = [a, b, c]
    probs = softmax(scores, scale=effective_scale)

    eps = 1e-12
    entropy = float(-(probs * np.log(probs + eps)).su
    top_idx = int(np.argmax(probs))
    top_prob = float(probs[top_idx])
```

@Add Context 8.9%

Plan, @ for context, / for commands

Agent Ctrl+I Auto

作業三 softmax模型 Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

6) 互動式觀察：三個輸入值與等效scale (包含scale與temperature)

```
def visualize_softmax(a: float, b: float, c: float, effective_scale: float):
    scores = [a, b, c]
    probs = softmax(scores, scale=effective_scale)

    eps = 1e-12
    entropy = float(-(probs * np.log(probs + eps)).sum())
    top_idx = int(np.argmax(probs))
    top_prob = float(probs[top_idx])

    header = HTML(value="<b>Input scores</b> → " + str(np.round(scores, 4)))
    smx = HTML(value="<b>Softmax</b> → " + str(np.round(probs, 6)) + f" (sum={probs.sum():.6f})")
    metrics = HTML(value=f"<b>Metrics</b> → entropy={entropy:.4f}, top={top_idx+1}, top_prob={top_prob:.4f}")

    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), dpi=120)

    # Left: bar chart at current effective scale
    labels = ["x1", "x2", "x3"]
    colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]
    axes[0].bar(labels, probs, color=colors)
    axes[0].set_ylim(0.0, 1.0)
    axes[0].set_title(f"Softmax at effective_scale = {effective_scale:.3f}")
    for i, v in enumerate(probs):
        axes[0].text(i, min(0.98, v + 0.03), f"{v:.3f}", ha="center")

    # Right: probability vs effective scale curve
    eff_min, eff_max = 0.01, 10.0
    eff_grid = np.linspace(eff_min, eff_max, 300)
    prob_curves = np.array([softmax(scores, s) for s in eff_grid])

    axes[1].plot(eff_grid, prob_curves[:, 0], color=colors[0], label="P(x1)")
    axes[1].plot(eff_grid, prob_curves[:, 1], color=colors[1], label="P(x2)")
```

[31] 0 秒

狀態 終端機 晚上9:45 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

```
    axes[1].plot(eff_grid, prob_curves[:, 2], color=colors[2], label="P(x3)")
    axes[1].axvline(effective_scale, color="#444444", linestyle="--", linewidth=1)
    axes[1].set_xlim(eff_min, eff_max)
    axes[1].set_ylim(0.0, 1.0)
    axes[1].set_xlabel("effective scale = scale × (1 / temperature)")
    axes[1].set_ylabel("probability")
    axes[1].set_title("Probability vs. effective scale")
    axes[1].legend(loc="best", frameon=False)

    plt.tight_layout()
    plt.show()

    display(VBox([header, smx, metrics]))


# Widgets
a_box = FloatText(value=1.0, description="x1:", step=0.1)
b_box = FloatText(value=2.0, description="x2:", step=0.1)
c_box = FloatText(value=3.0, description="x3:", step=0.1)

scale_slider = FloatSlider(
    value=1.0, min=0.01, max=10.0, step=0.01,
    description="scale",
    readout_format=".2f",
    continuous_update=False,
)

temp_slider = FloatSlider(
    value=1.0, min=0.01, max=10.0, step=0.01,
    description="temperature",
```

[31] 0 秒

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 程式碼 文字 全部執行

```
def compute_and_visualize(a: float, b: float, c: float, scale_value: float, temperature_value: float):
    eps = 1e-6
    scale_val = max(eps, float(scale_value))
    temp_val = max(eps, float(temperature_value))
    effective_scale = scale_val * (1.0 / temp_val)

    visualize_softmax(a, b, c, effective_scale)

    display(HTML(
        f"<b>scale</b> = {scale_val:.4f}  |  "
        f"<b>temperature</b> = {temp_val:.4f}  →  "
        f"<b>effective scale</b> = {effective_scale:.4f}"
    ))

ui = VBox([
    HBox([a_box, b_box, c_box]),
    HBox([scale_slider, temp_slider]),
])

out = widgets.interactive_output(
    compute_and_visualize,
    {
        "a": a_box,
        "b": b_box,
        "c": c_box,
        "scale_value": scale_slider,
        "temperature_value": temp_slider,
    }
)

display(ui, out)
```

x1: 2 x2: 3 x3: 4

scale 1.00 temperature 2.00

變數 終端機 晚上9:45 Python 3

作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

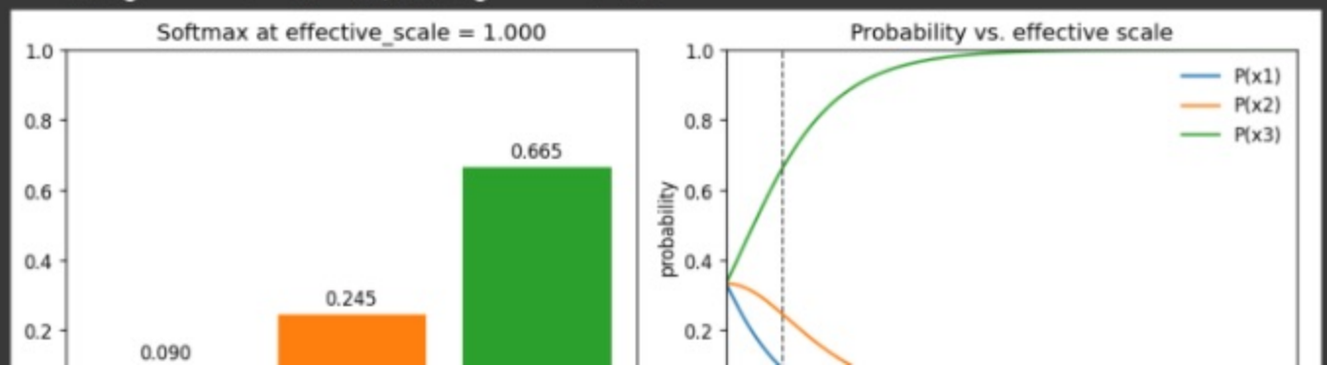
指令 程式碼 文字 全部執行

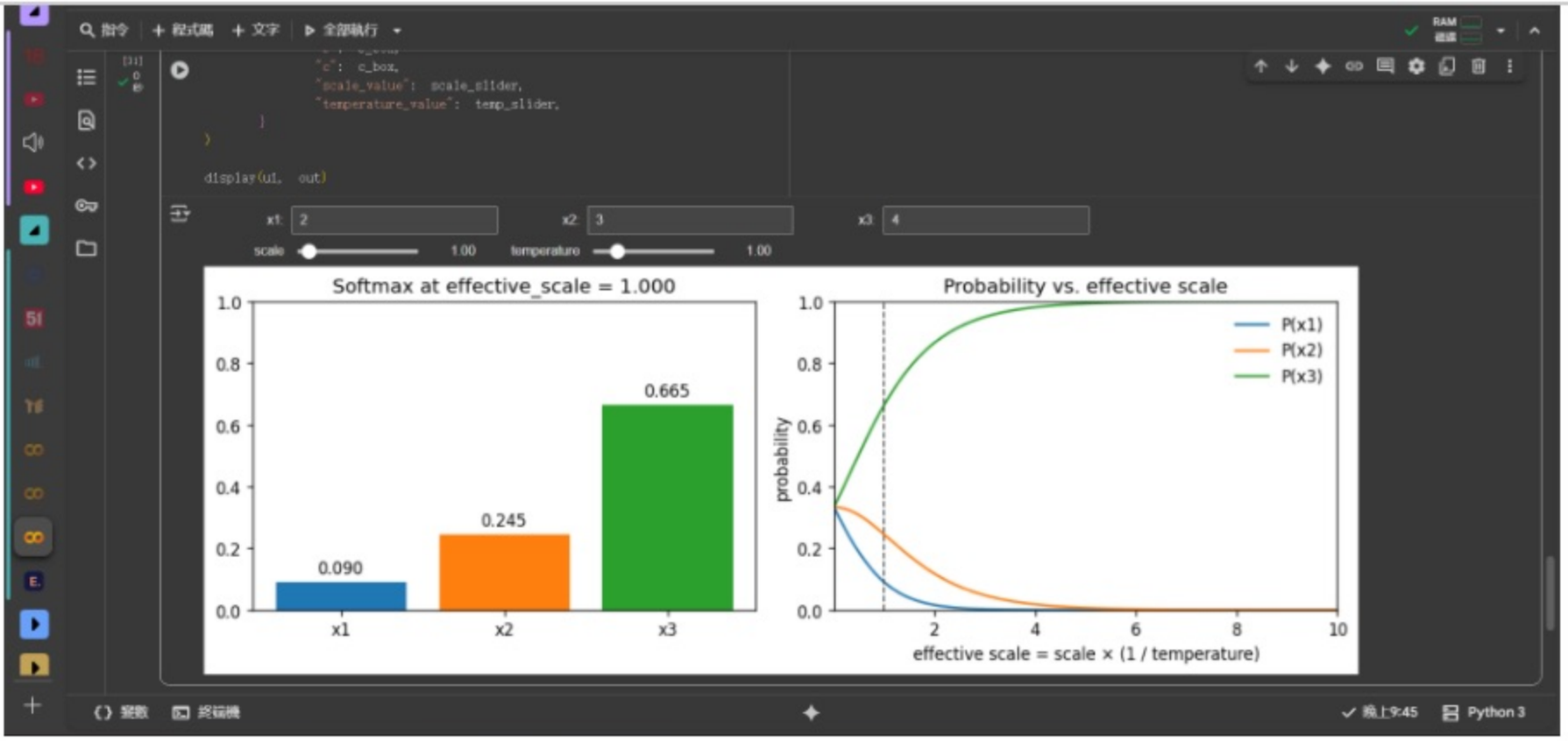
```
    "c": c_box,
    "scale_value": scale_slider,
    "temperature_value": temp_slider,
}
)

display(ui, out)
```

x1: 2 x2: 3 x3: 4

scale 1.00 temperature 1.00



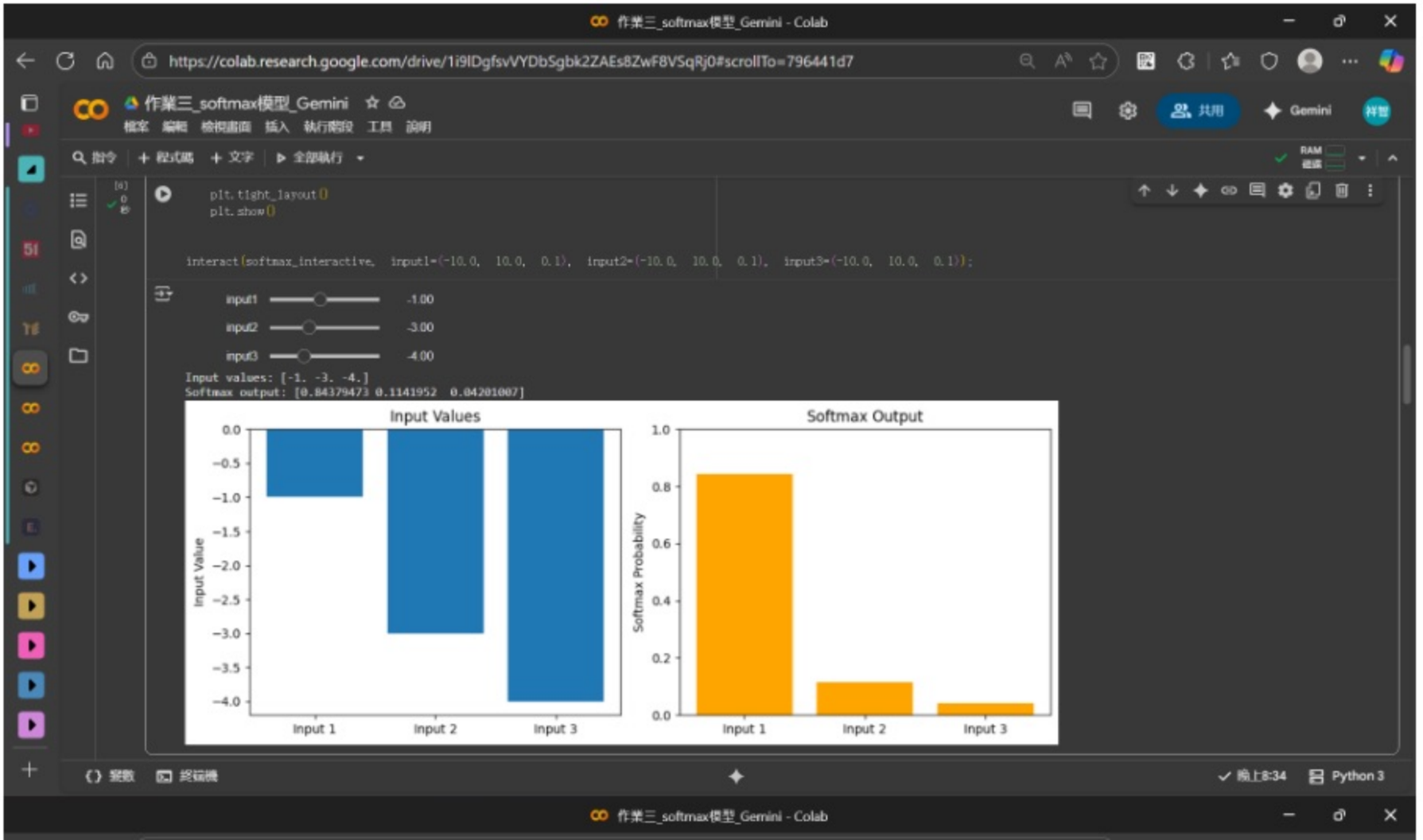


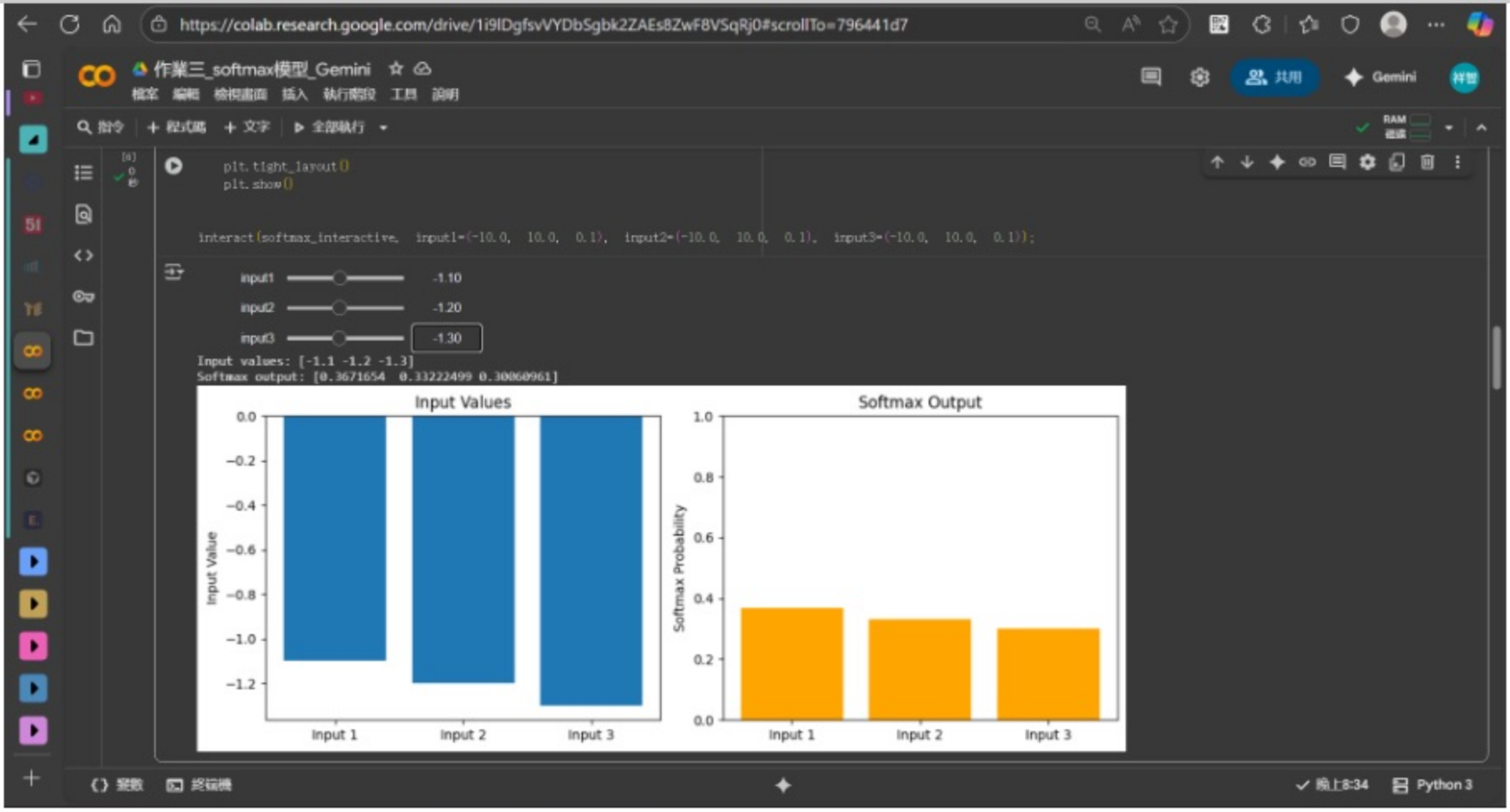
AI模型分析:

Cursor的程式碼在 softmax 函數的實現上更為進階（包含數值穩定性(`scaled - max_val以防止輸入值過大時可能發生的溢位問題`)和 scale 參數），並提供了更詳細且互動性更強的視覺化，特別強調了 scale 參數對機率分佈的影響(相同prompt的條件下)，並在不同 scale 範圍內更清楚地展示了「贏者通吃」的概念(尖銳程度)，另外(`List[float]`, `np.ndarray`)亦提高了程式碼的可讀性和可維

◦Gemini則提供了標準 softmax 計算和視覺化的基本互動式示範，在markdowns的編排與視覺呈現上略勝Cursor一籌，為適合展示和示範用的程式碼；Cursor則是更加注重程式的實用性與穩定性，適合在較複雜且對細節要求較高的程式系統中套入。

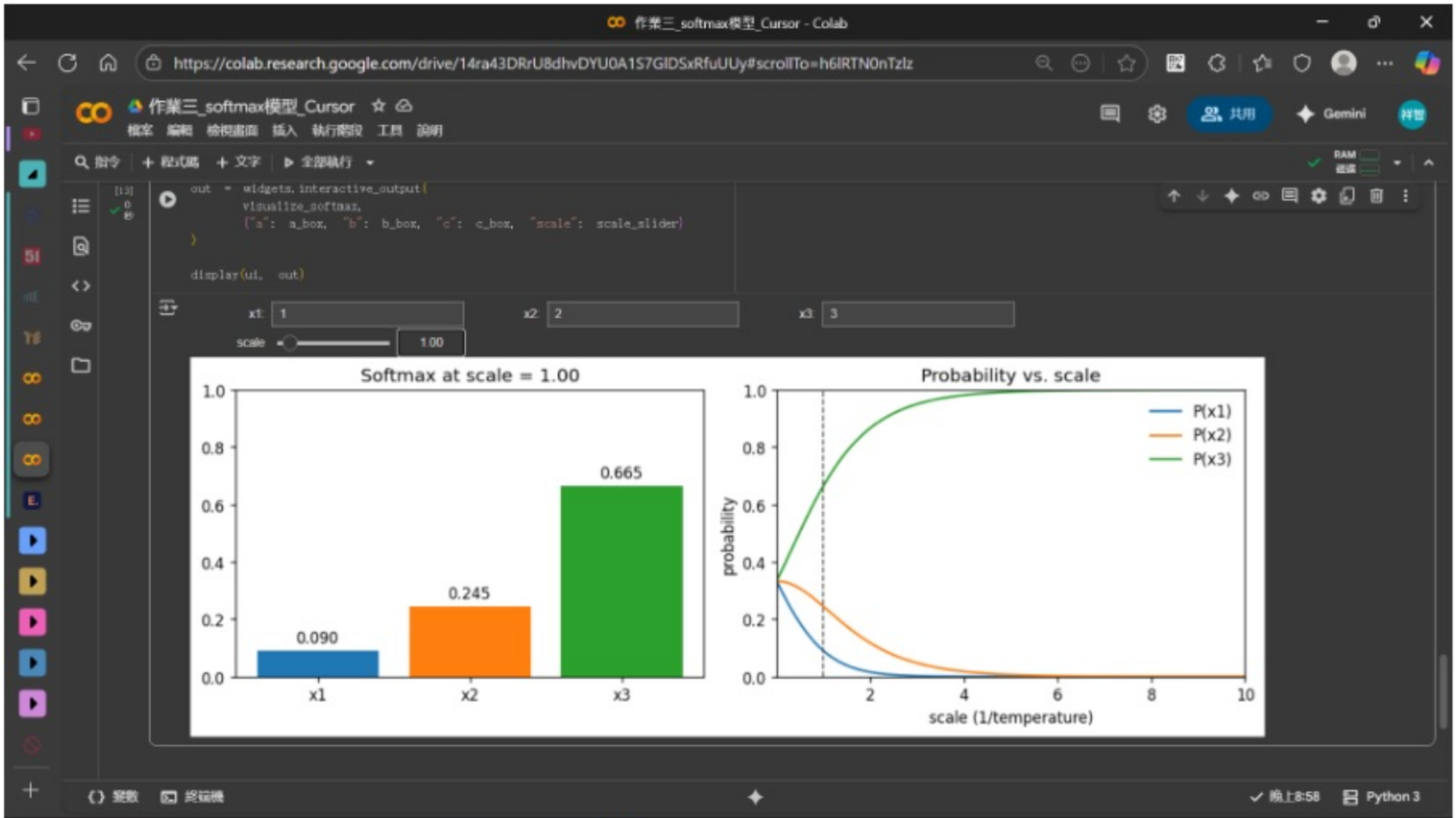
Softmax函數分析:

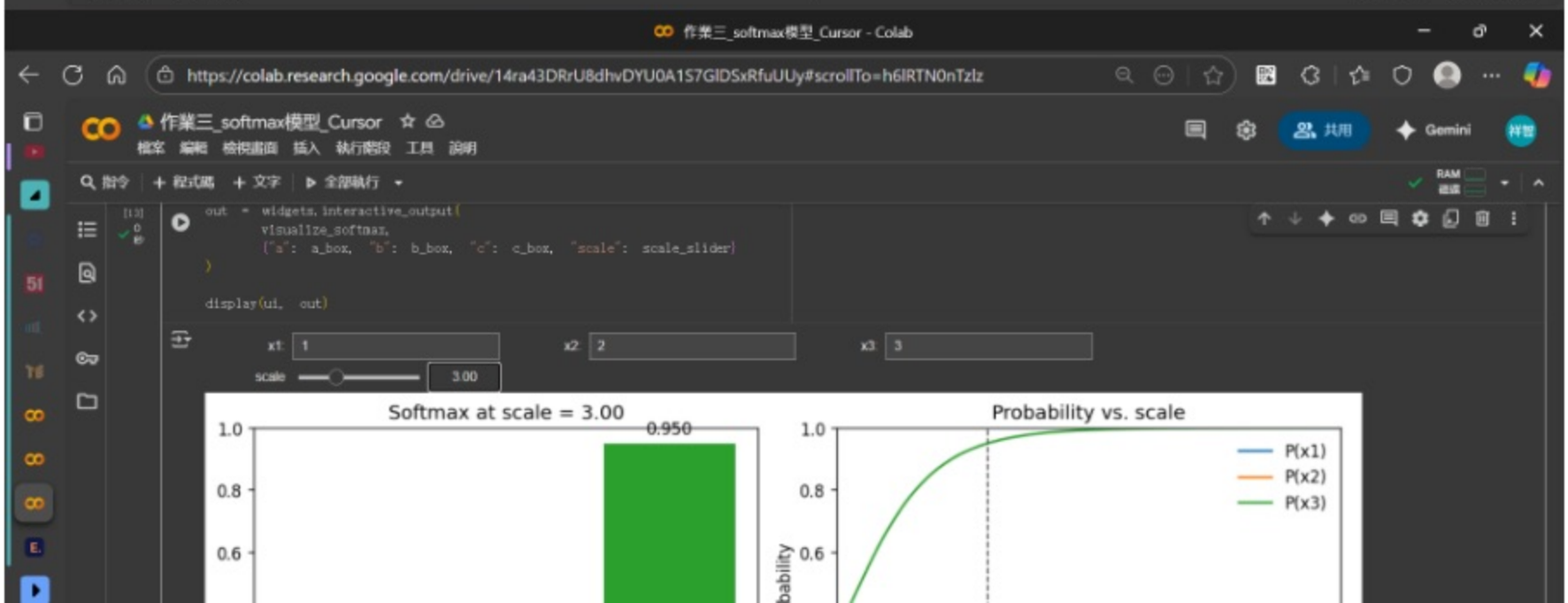




如上二圖所示，三個輸入值之間的差距越大，受到自然對數的指數放大倍率影響，機率差距將會更大，所謂的贏者通吃即是指輸入值的差距拉大時，機率將完全偏重數值最大者，使得引用此函數模型產出結果的傾向越單一確定。而因為輸入值是函數中的指數自變數，所以當輸入值為負時，其輸出值大小將會變為該自變數取絕對值的輸出值倒數，也就是當輸入值負越大則機率越小；負越小則機率越大。

大。


作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=h6IRTN0nTzIz
作業三_softmax模型_Cursor
out = widgets.interactive_output(
visualize_softmax,
{"a": a_box, "b": b_box, "c": c_box, "scale": scale_slider}
)
display(ui, out)
x1: 1
x2: 2
x3: 3
scale: 1.00
Softmax at scale = 1.00
0.090
0.245
0.665
x1
x2
x3
Probability vs. scale
P(x1)
P(x2)
P(x3)
probability
scale (1/temperature)

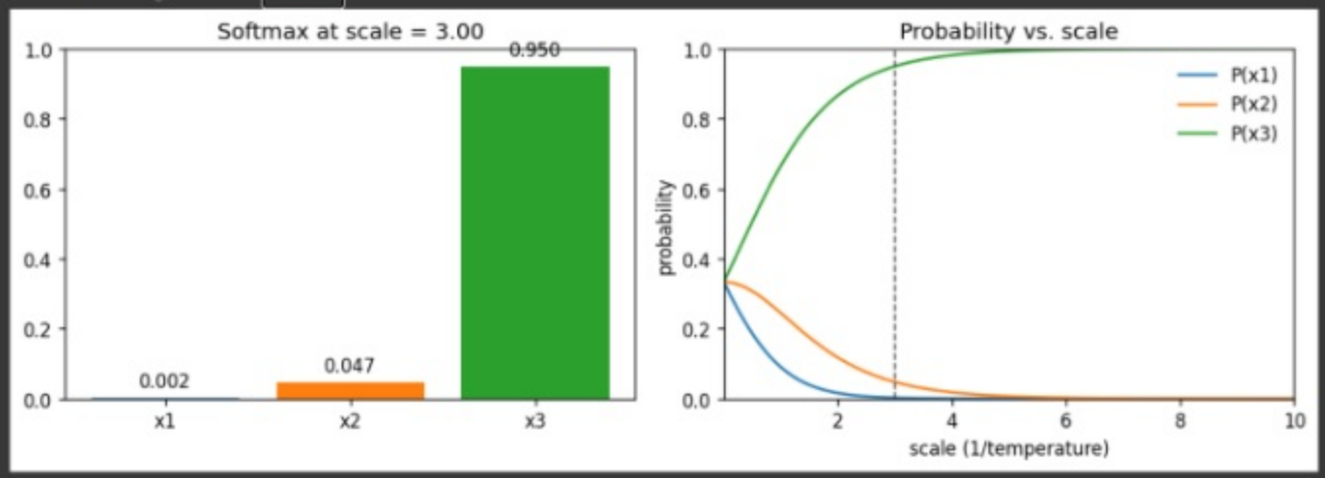

作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=h6IRTN0nTzIz
作業三_softmax模型_Cursor
out = widgets.interactive_output(
visualize_softmax,
{"a": a_box, "b": b_box, "c": c_box, "scale": scale_slider}
)
display(ui, out)
x1: 1
x2: 2
x3: 3
scale: 3.00
Softmax at scale = 3.00
0.950
Probability vs. scale
P(x1)
P(x2)
P(x3)

```
out = widgets.interactive_output(
    visualize_softmax,
    {"a": a_box, "b": b_box, "c": c_box, "scale": scale_slider}
)

display(ui, out)
```

x1: 1 x2: 2 x3: 3

scale: 3.00



作業三_softmax模型_Cursor - Colab

https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=h6IRTN0nTzIz

作業三_softmax模型_Cursor

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

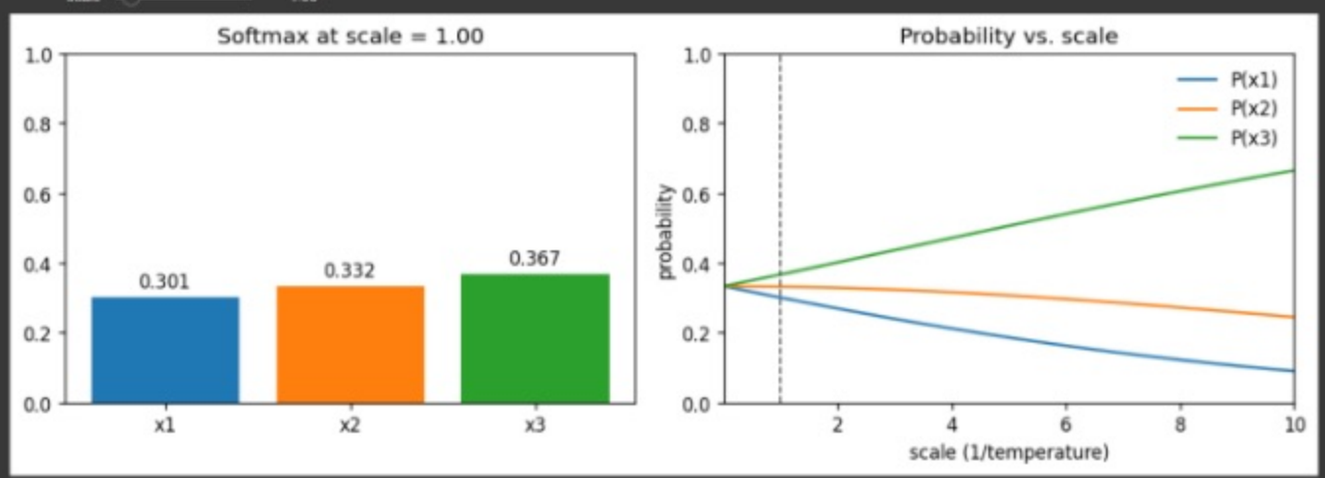
指令 + 程式碼 + 文字 全部執行

```
out = widgets.interactive_output(
    visualize_softmax,
    {"a": a_box, "b": b_box, "c": c_box, "scale": scale_slider}
)

display(ui, out)
```

x1: 1.1 x2: 1.2 x3: 1.3

scale: 1.00



變數 終端機

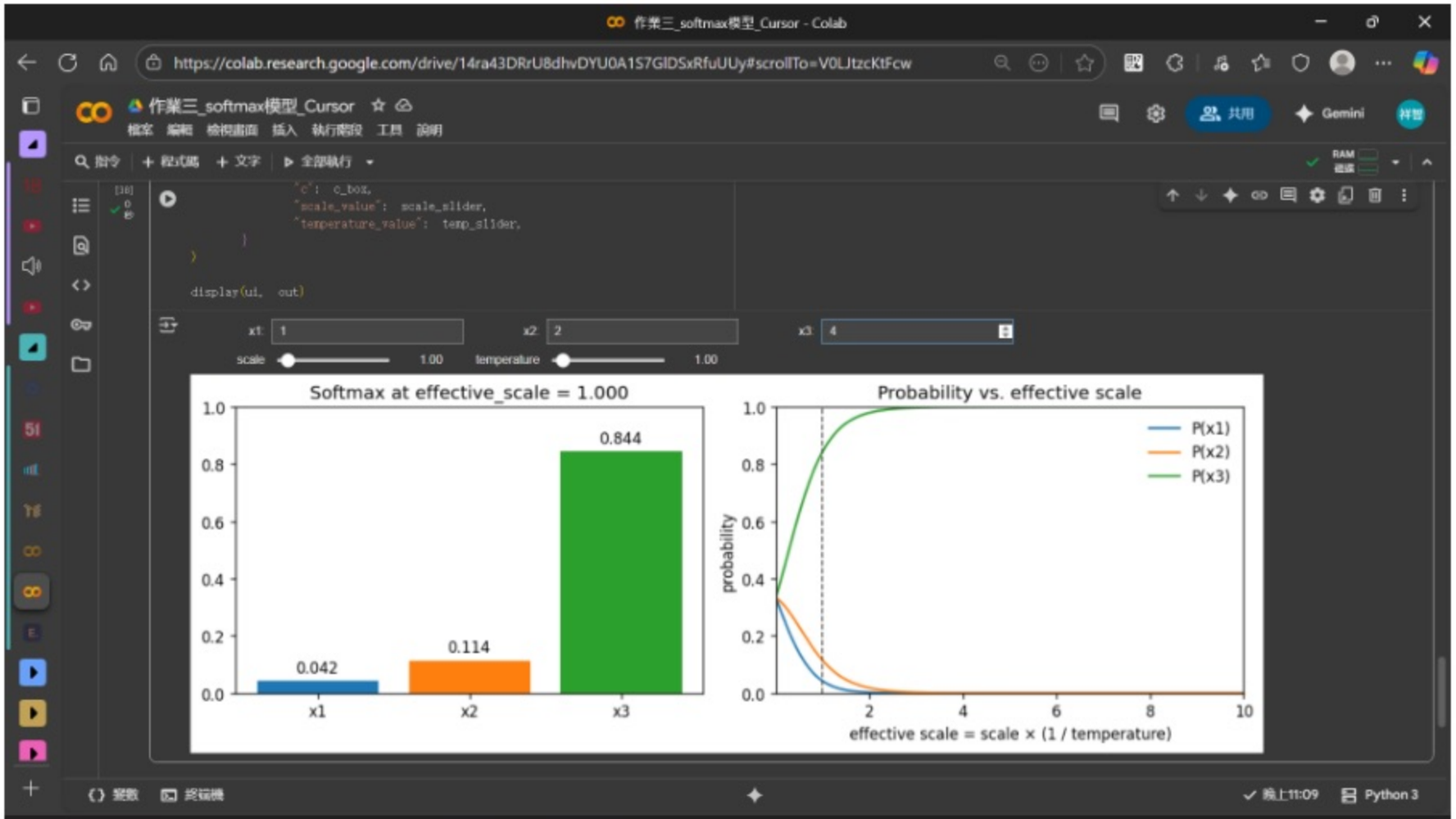
晚上8:58 Python 3

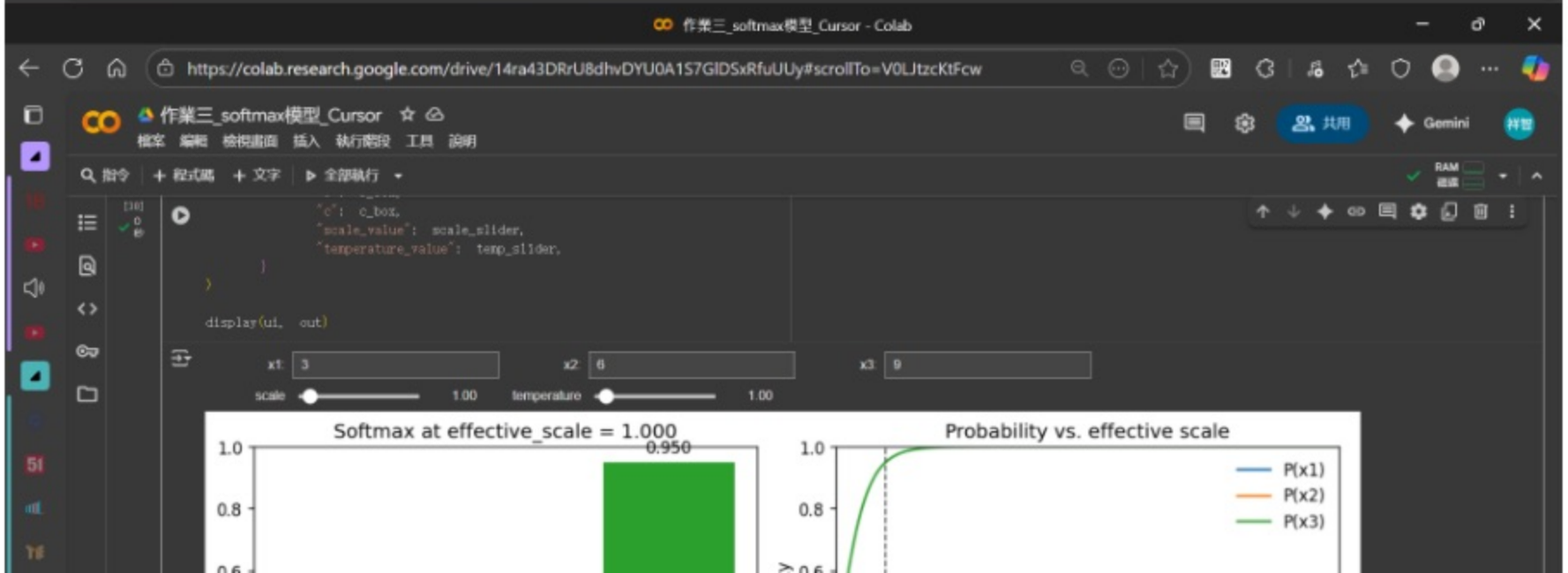
作業三_softmax模型_Cursor - Colab

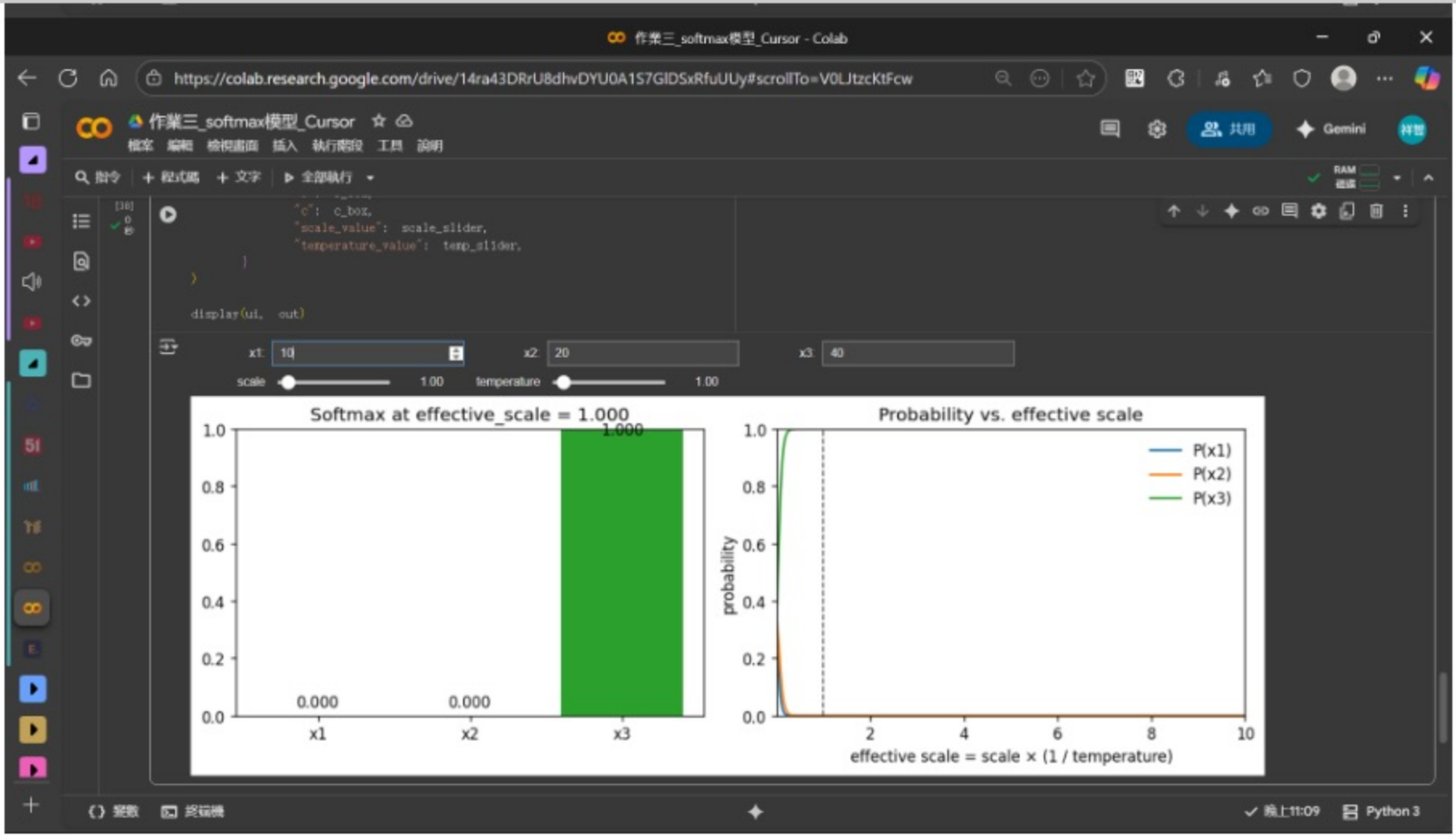


而當函數加入了scale與temperature參數調整後會發現，由於scale是輸入值的乘項參數；temperature是輸入值的除項參數，所以當scale的數值越大，輸入值的差距反映在機率質量函數的輸出差距將會進一步擴大(變成指數的乘數內積倍率)，而scale反映在機率質量函數的曲線圖形變化則是跟輸入值的差距有關，若輸入值等差或比例相同則圖形會較相似甚至機率分布完全一樣，但僅限於數值差距小的尺度下(-10~10的整數)。

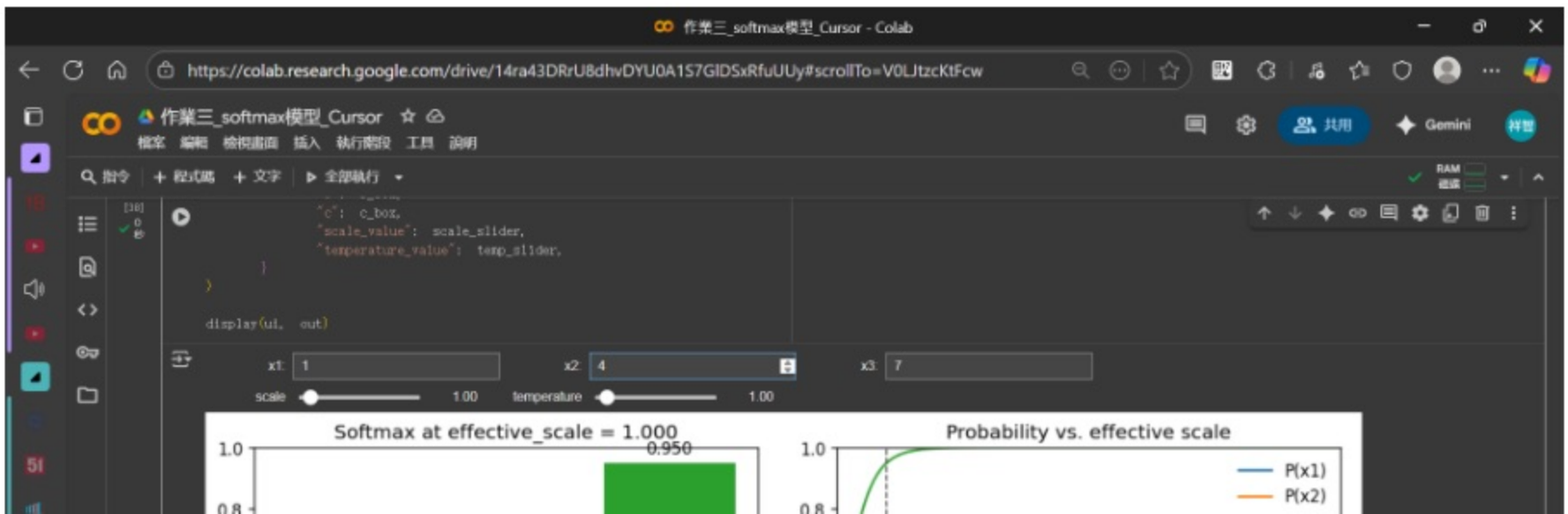
EX:差2倍:

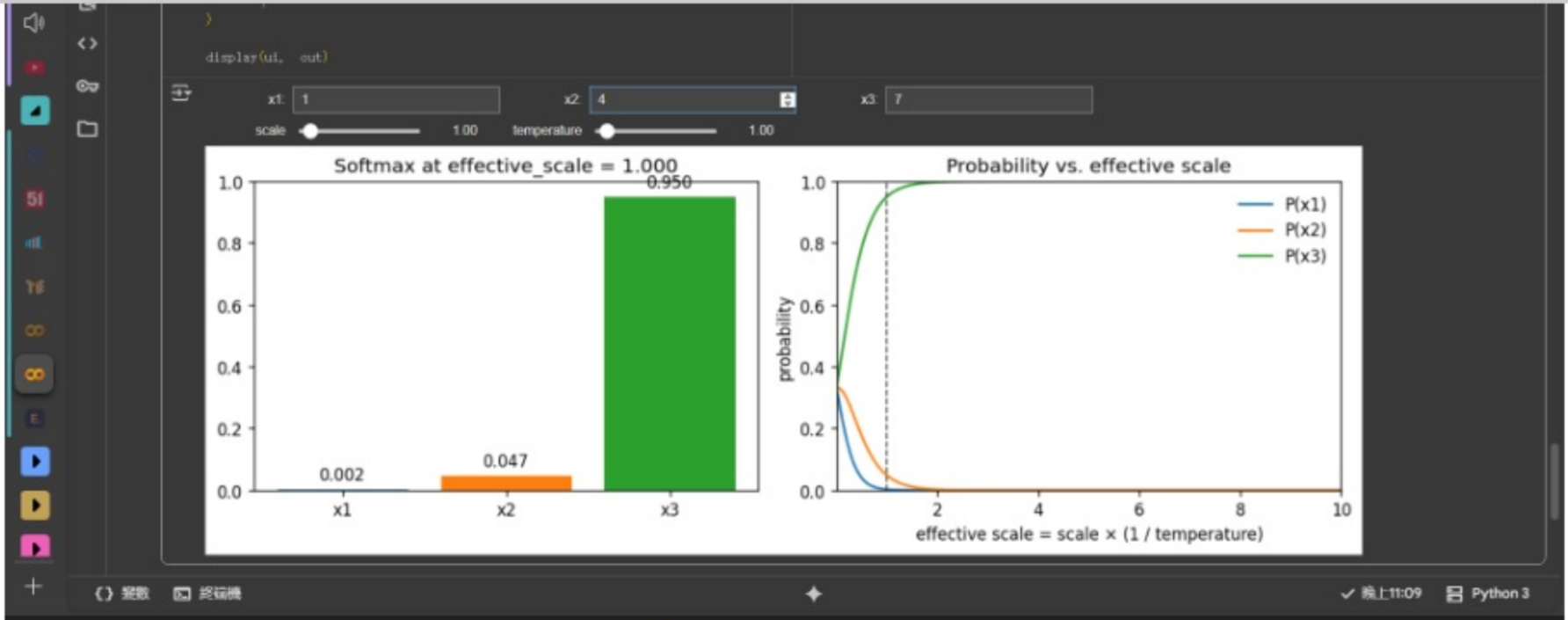

作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhwDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw
作業三_softmax模型_Cursor
檔案 編輯 檢視畫面 插入 執行階段 工具 說明
共用
Gemini
"c": c_box,
"scale_value": scale_slider,
"temperature_value": temp_slider,
display(ui, out)
x1: 1
x2: 2
x3: 4
scale 1.00
temperature 1.00
Softmax at effective_scale = 1.000
0.042
0.114
0.844
x1
x2
x3
Probability vs. effective scale
P(x1)
P(x2)
P(x3)
probability
effective scale = scale × (1 / temperature)
Python 3

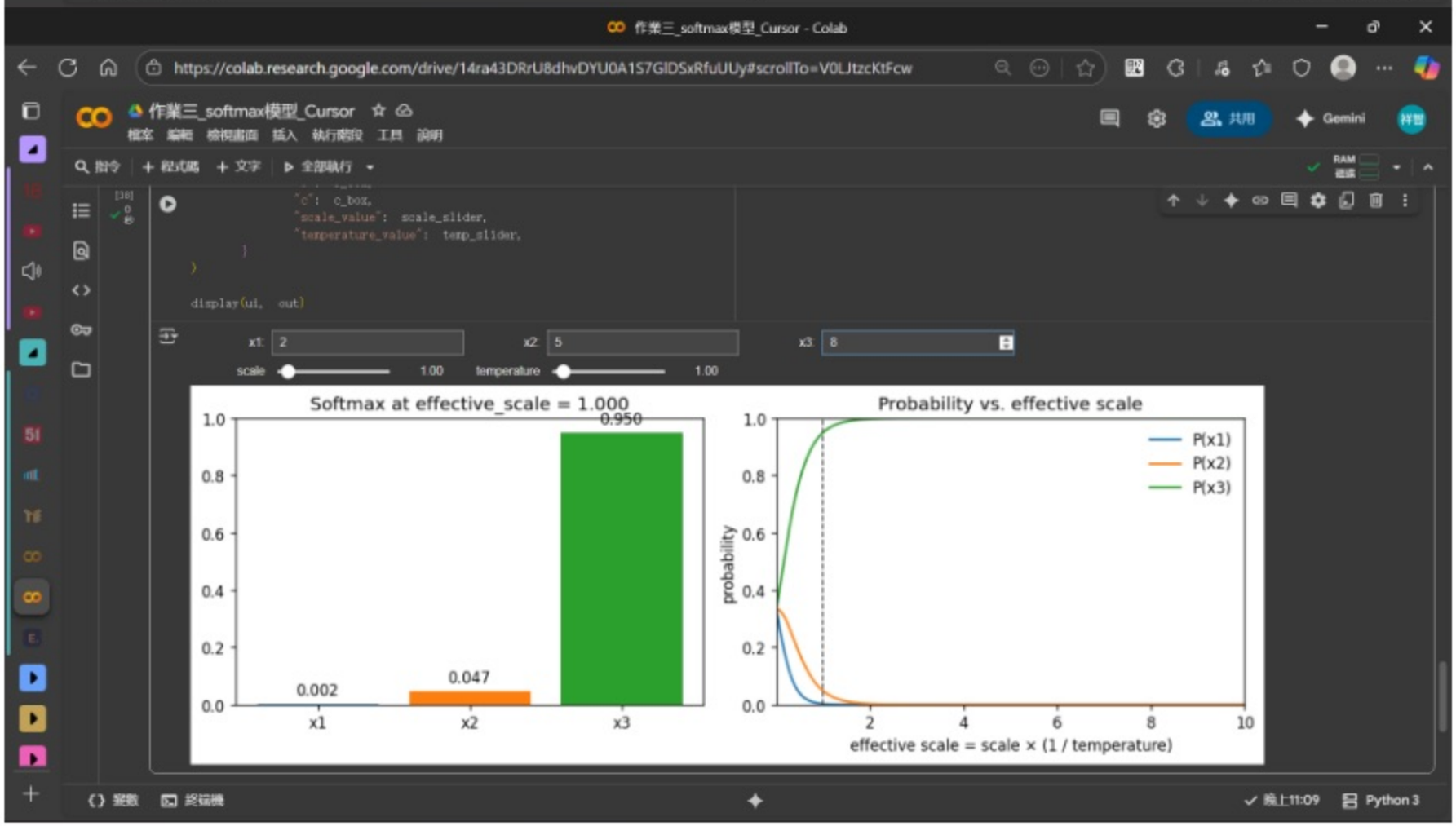

作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhwDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw
作業三_softmax模型_Cursor
檔案 編輯 檢視畫面 插入 執行階段 工具 說明
共用
Gemini
"c": c_box,
"scale_value": scale_slider,
"temperature_value": temp_slider,
display(ui, out)
x1: 3
x2: 6
x3: 9
scale 1.00
temperature 1.00
Softmax at effective_scale = 1.000
0.950
Probability vs. effective scale
P(x1)
P(x2)
P(x3)


作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw
作業三_softmax模型_Cursor
"c": c_box,
"scale_value": scale_slider,
"temperature_value": temp_slider,
display(ui, out)
x1: 10
x2: 20
x3: 40
scale 1.00
temperature 1.00
Softmax at effective_scale = 1.000
1.000
0.000
0.000
x1
x2
x3
Probability vs. effective scale
P(x1)
P(x2)
P(x3)
probability
effective scale = scale × (1 / temperature)
Python 3

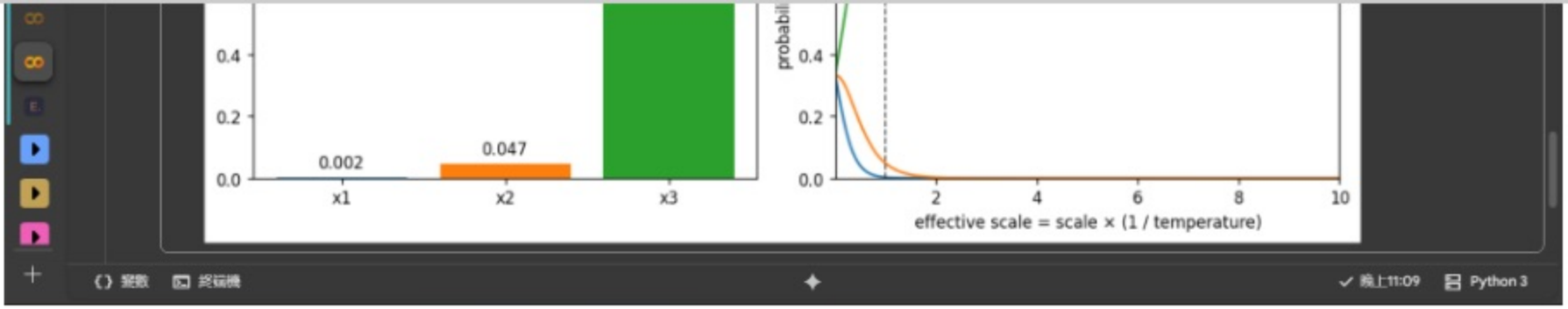
EX:等差:


作業三_softmax模型_Cursor - Colab
https://colab.research.google.com/drive/14ra43DRrU8dhvDYU0A1S7GIDSxRfuUUy#scrollTo=V0LJtzcKtFcw
作業三_softmax模型_Cursor
"c": c_box,
"scale_value": scale_slider,
"temperature_value": temp_slider,
display(ui, out)
x1: 1
x2: 4
x3: 7
scale 1.00
temperature 1.00
Softmax at effective_scale = 1.000
0.950
Probability vs. effective scale
P(x1)
P(x2)





EX:缩小尺度至差距小數點後，圖形明顯不同且發散幅度



EX:縮小尺度至差距小數點後，圖形明顯不同且發散幅度較小:

